Notas de Aula
Prof. Adriano Milanez

Programação Web I
Responsive Web Development
Front-End Design Engineering
Programação Web Responsiva

[aula 19]
[Atualizado em: 18/09/2025]

[VITE – HOOKS]

VITE – HOOKS

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **Hooks** no React são funções especiais que permitem adicionar funcionalidades a componentes funcionais, como gerenciamento de estado e outros recursos do React.

- ✓ Quando utilizados com TypeScript, oferecem tipagem segura, garantindo previsibilidade nos valores do estado e nos retornos das funções.

- ✓ Além disso, possibilitam que componentes funcionais tenham capacidades equivalentes às dos componentes de classe, sem a necessidade de usar classes.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **Por que usar Hooks?**
  - Utilizar Hooks no React traz diversas vantagens, tornando o desenvolvimento mais organizado, eficiente e moderno.
  - Aqui estão alguns motivos principais:
    - **Eliminação de Componentes de Classe**: Permitem usar estado e ciclos de vida em componentes funcionais, tornando o código mais simples e legível.
    - **Melhor Reutilização de Lógica**: Com useCustomHooks, facilita a extração e reutilização de lógica entre componentes.
    - **Organização e Clareza**: Evita métodos de ciclo de vida complexos, tornando o código mais modular e fácil de manter.
    - **Melhor Performance**: Reduz renderizações desnecessárias e otimiza atualizações de estado.
    - **Menos Código Boiler-plate**: Diminui a necessidade de usar classes, this, bind, e construtores.
    - **Facilidade de Testes**: Componentes funcionais com hooks são mais fáceis de testar do que componentes de classe.
    - **Melhor Compatibilidade com React Moderno**: Hooks são a abordagem recomendada para novos projetos e atualizações.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **Quais são os Hooks no React?**
  - **useCallback** – Memoriza funções para evitar recriações desnecessárias.
  - **useContext** – Acessa valores de um contexto global.
  - **useDebugValue** – Adiciona rótulos personalizados no React DevTools para depuração.
  - **useDeferredValue** – Atraso na atualização de um estado para evitar re-renders desnecessários.
  - **useEffect** – Executa efeitos colaterais.
  - **useId** – Gera identificadores únicos para elementos, útil em listas ou formulários.
  - **useImperativeHandle** – Permite expor métodos específicos quando usado com forwardRef.
  - **useLayoutEffect** – Semelhante ao useEffect, mas executa antes da renderização.
  - **useMemo** – Memoriza valores computados para otimizar performance.
  - **useReducer** – Gerencia estados mais complexos com lógica baseada em ações.
  - **useRef** – Mantém referências a elementos ou valores sem causar re-render.
  - **useState** – Gerencia o estado local.
  - **useSyncExternalStore** – Gerencia assinaturas de estados externos, útil para bibliotecas como Redux.
  - **useTransition** – Garante transições suaves em mudanças de estado, mantendo a interface responsiva.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **Quais são os Hooks vamos trabalhar?**
  - ✓ **useState** – Gerencia o estado local.
  - ✓ **useEffect** – Executa efeitos colaterais.
  - ✓ **useContext** – Acessa valores de um contexto global.
  - ✓ **useCallback** – Memoriza funções para evitar recriações desnecessárias.
  - ✓ **useMemo** – Memoriza valores computados para otimizar performance.
  - ✓ **useDebugValue** – Adiciona rótulos personalizados no React DevTools para depuração.
  - ✓ **useDeferredValue** – Atraso na atualização de um estado para evitar re-renders desnecessários.
  - ✓ **useId** – Gera identificadores únicos para elementos, útil em listas ou formulários.
  - ✓ **useImperativeHandle** – Permite expor métodos específicos quando usado com forwardRef.
  - ✓ **useLayoutEffect** – Semelhante ao useEffect, mas executa antes da renderização.
  - ✓ **useReducer** – Gerencia estados mais complexos com lógica baseada em ações.
  - ✓ **useRef** – Mantém referências a elementos ou valores sem causar re-render.
  - ✓ **useSyncExternalStore** – Gerencia assinaturas de estados externos, útil para bibliotecas como Redux.
  - ✓ **useTransition** – Garante transições suaves em mudanças de estado, mantendo a interface responsiva.

2

REACT – HOOKS – USESTATE

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useState**

  - O **`useState`** é um hook do React que permite adicionar e gerenciar estado em componentes funcionais. Antes do React Hooks, apenas componentes de classe podiam ter estado. Com **`useState`**, qualquer componente funcional pode armazenar valores e atualizá-los.

  - O **`useState`** é uma função que retorna dois valores: o estado atual e uma função para atualizar esse estado.

  - A função **`useState`** aceita um argumento opcional, que é o valor inicial do estado.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useState – SINTAXE**

  - `const [estado, setEstado] = useState(valorInicial);`

    - **`estado`**: variável que armazena o valor atual.
    - **`setEstado`**: função usada para atualizar o valor.
    - **`valorInicial`**: o valor inicial do estado.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState – Características**

  - ✓ **Gerencia estado local** – Armazena valores que podem mudar ao longo do ciclo de vida do componente.
  - ✓ **Re-renderiza o componente** – Sempre que o estado muda, o componente é atualizado.
  - ✓ **Aceita qualquer tipo de valor** – Pode armazenar números, strings, arrays, objetos e até funções.
  - ✓ **Função de atualização segura** – Garante que o novo valor seja baseado no estado anterior.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useState – Como funciona?**
  - **O useState retorna dois valores:**
    - O estado atual.
    - Uma função para atualizar esse estado.
  - **A atualização do estado pode ser feita de duas formas:**
    - Atribuindo um novo valor diretamente `setEstado(novoValor)`.
    - Passando uma função que recebe o estado anterior `setEstado(prev => prev + 1)`.
    - Isso evita problemas quando o novo estado depende do valor anterior, garantindo que ele esteja atualizado corretamente.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useState**
  - **Exemplo 01 – Contador: Exibe modificação somente após clicar no botão**
    - Vamos criar novas rotas, para que consigamos navegar entre os exemplos que vamos criar, para isso vamos relembrar a aula anterior, instalar a biblioteca react-router-dom e criar novas rotas.

```
'npm install react-router-dom'
```

    - Modificar o arquivo App.tsx para criar nossas rotas.

A partir desta aula não mostrarei mais o passo para a criação dos projetos!

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 01 – Contador: Exibe modificação somente após clicar no botão**

```
1.  import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
2.  import UseState_Exemplo01 from './pages/useState/UseState_Exemplo01';
3.  const App = () => {
4.   return (
5.     <BrowserRouter>
6.      <div>
7.        {/* Menu de navegação */}
          <nav>
8.         <ul>
9.          <li><Link to="/" className="hover:underline">Home</Link></li>
10.         <li>UseState
11.           <ul>
12.            <li>
13.             <Link to="/useState_exemplo01" className="hover:underline">Exemplo01</Link>
14.            </li>
```

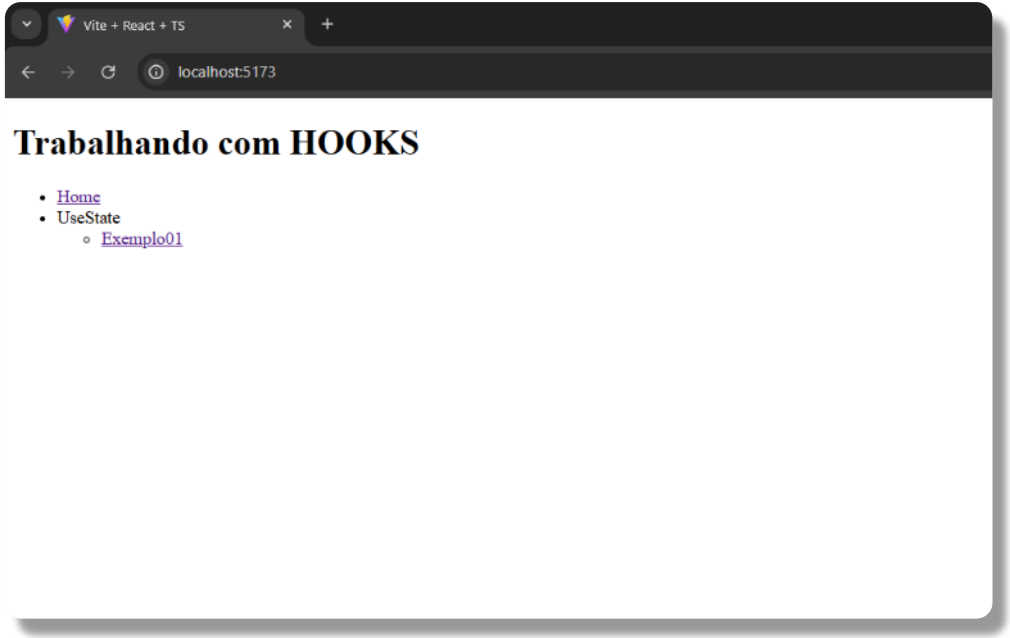
COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 01 – Contador: Exibe modificação somente após clicar no botão**

```
15.            </ul>
16.           </li>
17.         </ul>
18.       </nav>
19.       {/* Rotas */}
          <Routes>
20.         <Route path="/useState_exemplo01" element={<UseState_Exemplo01 />} />
21.       </Routes>
22.     </div>
23.   </BrowserRouter>
24.  );
25. };
26. export default App;
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 01 – Contador: Exibe modificação somente após clicar no botão**



COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

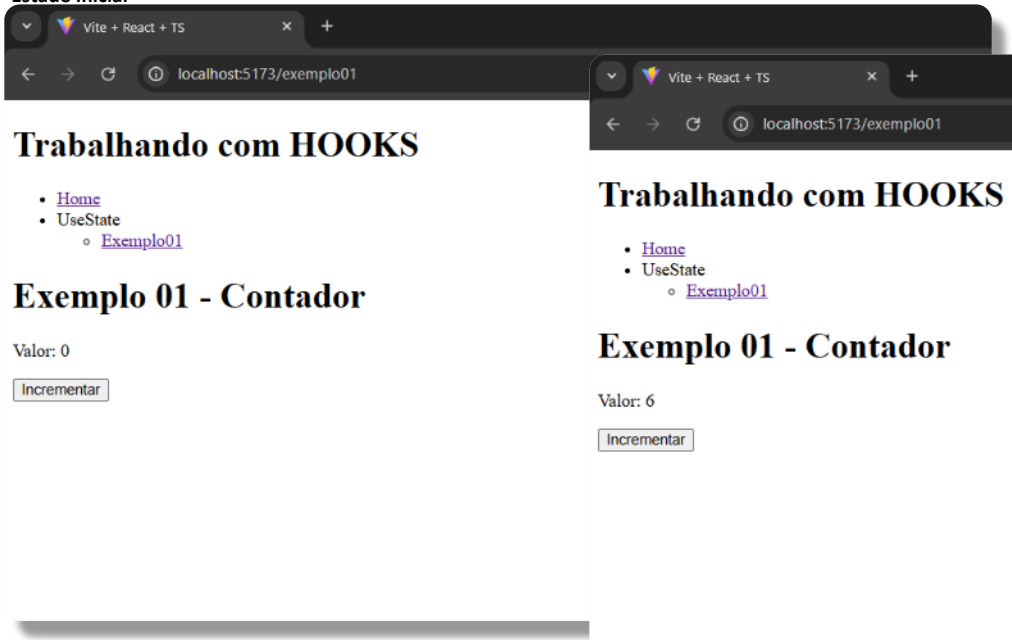
- ✓ **HOOKS – useState**
  - ✓ **Exemplo 01 – Contador: Exibe modificação somente após clicar no botão**

```
1.  import { useState } from "react"; // Importa o Hook useState do React
2.  const UseState_Exemplo01 = () => {
3.     // Declara um estado chamado "contador", inicializado com 0
       // "setContador1" é a função usada para atualizar esse estado
       const [contador, setContador] = useState(0);
4.     return (
5.      <>
6.       <h1>Exemplo 01 - Contador</h1>
7.       {/* Exibe o valor atual do contador */}
         <p>Valor: {contador}</p>
8.       {/* Botão que, ao ser clicado, incrementa o contador em +1 */}
         <button onClick={() => setContador(contador + 1)}>Incrementar</button>
9.      </>
10.    );
11. };
12. export default UseState_Exemplo01;
```

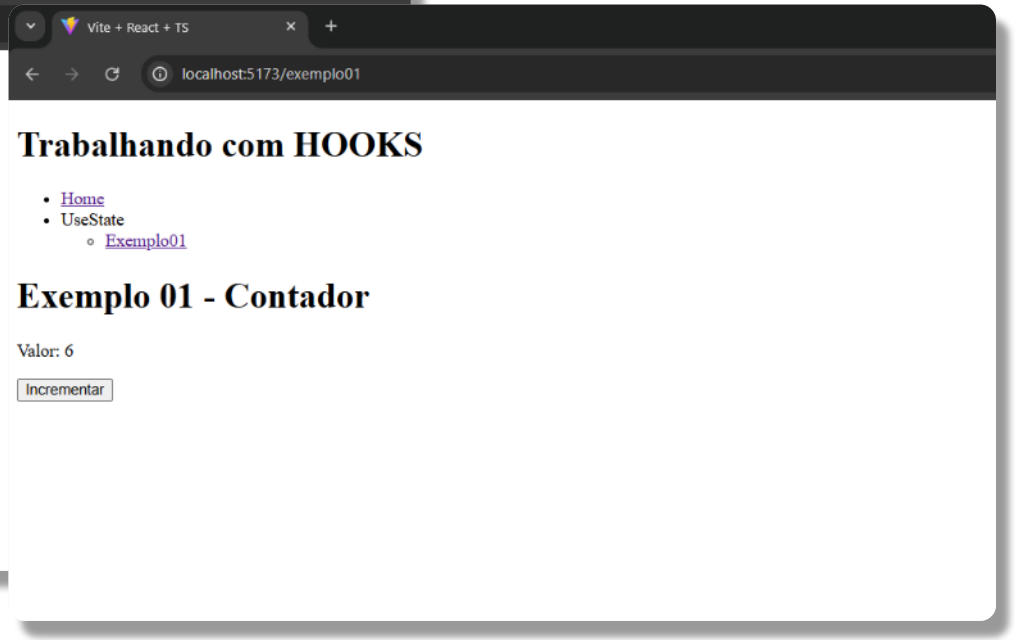
COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 01 – Contador: Exibe modificação somente após clicar no botão**

Estado Inicial



Estado Final



COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – A diretiva 'use Client'**

  - ✓ **O que é?**
    - ✓ O 'use client' é uma diretiva especial que indica que um componente deve ser renderizado no cliente (navegador), em vez do servidor.
    - ✓ Isso é necessário porque, por padrão, os componentes dentro da pasta app/ são Server Components e são processados no servidor.
  - ✓ **Quando utilizar?**
    - ✓ Se você estiver utilizando **NEXT.JS 13** ou superior, se estiver usando **Vite** (como no seu projeto atual), você não precisa se preocupar com 'use client' — pois **todos os componentes** são client-side por padrão.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 02 – Contador 02: Habilita/Desabilita botões**

```
1.  import { useState } from "react"; // Importa o Hook useState do React
2.  const UseState_Exemplo02 = () => {
3.   const [contador, setContador] = useState(0);
4.   const limite = 10; // Define o limite máximo do contador
5.   return (
6.     <>
7.       <h1>Exemplo 02 - Contador 02: Habilita/Desabilita botões</h1>
8.       <p>Valor: { contador }</p>
9.       <button onClick={() => setContador(contador + 1) } disabled={ contador >= limite }>Aumentar</button>
10.      {/* Botão de decremento, desativado quando atinge o valor 0 */}
         <button onClick={() => setContador( contador - 1 ) } disabled={ contador == 0 }>Diminuir</button>
11.      <button onClick={() => setContador(0) } >Resetar</button> {/* Botão para resetar o contador */}
12.    </>
13.  );
14. };
15. export default UseState_Exemplo02;
```

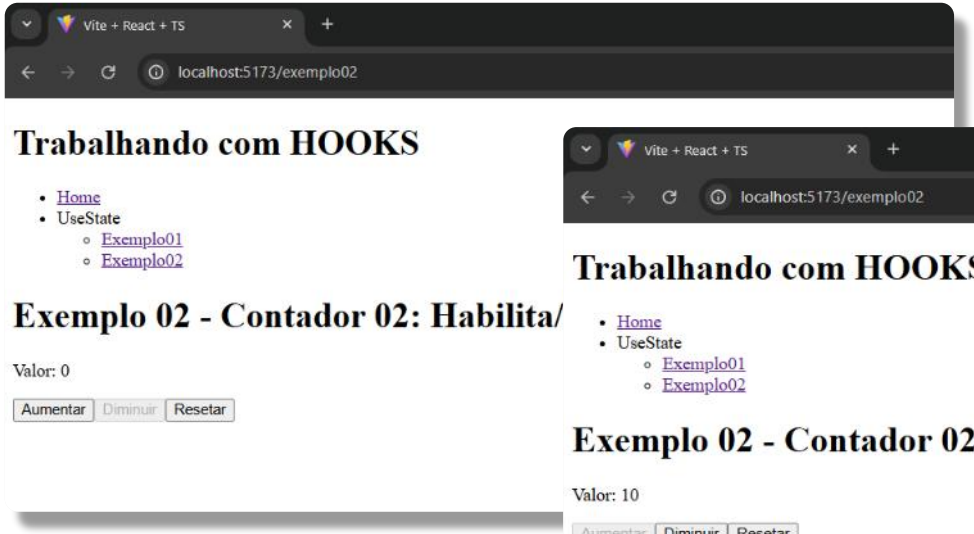
COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 02 – Contador 02: Habilita/Desabilita botões**
  - ✓ **Não esqueça de modificar o arquivo APP, acrescentando este e os demais componentes que serão criados nas rotas, conforme fizemos no Exemplo 01, slide 13.**

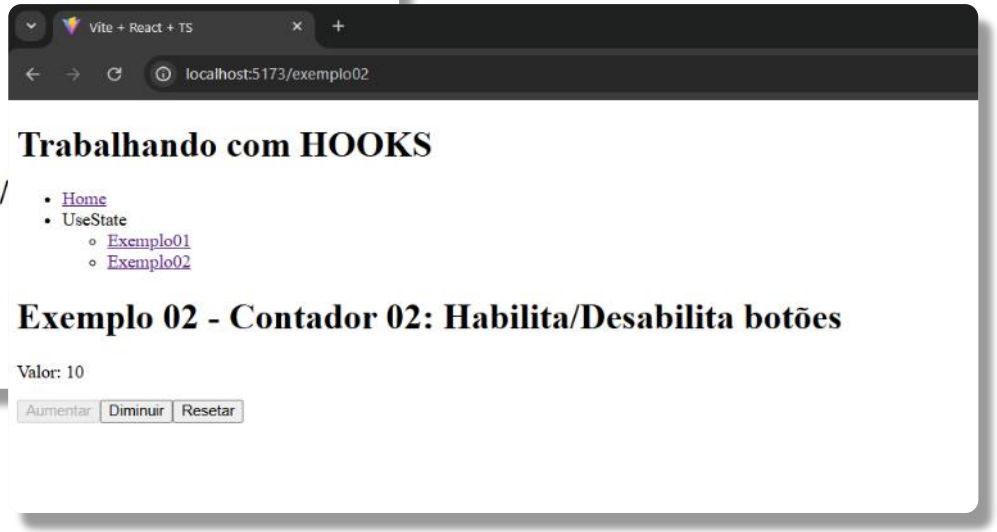
COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 02 – Contador 02: Habilita/Desabilita botões**

Estado Inicial



Estado Final



COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 03 – Formulário 01: Exibe modificação enquanto o valor é digitado no campo**

```
1.  import { useState } from "react"; // Importa o Hook useState do React
2.  const UseState_Exemplo03 = () => {
3.    // Inicializa o estado do formulário com uma string vazia
      const [nome, setNome] = useState("");
4.    // Função para lidar com a mudança no input
      const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
5.      setNome(event.target.value);
6.    };
7.    return (
8.      <>
9.        <h1>Exemplo 03 - Formulário 01: Exibe modificação enquanto o valor é digitado no
          campo</h1>
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 03 – Formulário 01: Exibe modificação enquanto o valor é digitado no campo**

```
10.      <form>
11.        <label>Nome: <input type="text"
12.          // O valor do campo é controlado pelo estado
             value={ nome }
13.          // Atualiza o estado a cada mudança
             onChange1={ handleChange } />
14.        </label>
15.        {/* Exibe o nome digitado */}
           <p>Nome digitado: { nome }</p>
16.      </form>
17.    </>
18.  );
19. };
20. export default UseState_Exemplo03;
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 03 – Formulário 01: Exibe modificação enquanto o valor é digitado no campo**

Estado Inicial

Vite + React + TS

localhost:5173/exemplo03

Trabalhando com HOOKS

- Home
- UseState
  - Exemplo01
  - Exemplo02
  - Exemplo03

Exemplo 03 - Formulário 01: Exibe mo

Nome:

Nome digitado:

Estado Final

Vite + React + TS

localhost:5173/exemplo03

Trabalhando com HOOKS

- Home
- UseState
  - Exemplo01
  - Exemplo02
  - Exemplo03

Exemplo 03 - Formulário 01: Exibe modificação enquanto o v

Nome: Darth Vader

Nome digitado: Darth Vader

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 04 – Formulário 02: Exibe modificação somente após clicar no botão**

```
1.  import { useState } from "react"; // Importa o Hook useState do React
2.  const UseState_Exemplo04 = () => {
3.    const [nome, setNome] = useState(""); // Estado para armazenar o nome digitado
4.    const [nomeExibido, setNomeExibido] = useState(""); // Estado para exibir o nome após o clique
5.    // Atualiza o estado conforme o usuário digita
      const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
6.      setNome(event.target.value);
7.    };
8.    // Atualiza o nome que será exibido ao clicar no botão
      const handleSubmit = () => {
9.      setNomeExibido(nome);
10.     setNome("");
11.   };
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 04 – Formulário 02: Exibe modificação somente após clicar no botão**

```
12.  return (
13.    <>
14.      <h1>Exemplo 04 - Formulário 02: Exibe modificação somente após clicar no botão</h1>
15.      <form onSubmit={(e) => e.preventDefault()}>
16.        <label>Nome:
17.          // Atualiza o estado ao digitar
             <input type="text" value={ nome } onChange={ handleChange } />
18.        </label>
19.        <button type="button" onClick={ handleSubmit }>Mostrar Nome</button>
20.      </form>
21.      {/* Exibe o nome apenas após o botão ser clicado */}
         { nomeExibido && <p><b>Nome digitado</b>: { nomeExibido }</p> }
22.    </>
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 04 – Formulário 02: Exibe modificação somente após clicar no botão**

```
23.   );
24.  }
25.  export default UseState_Exemplo04;
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState**
  - ✓ **Exemplo 04 – Formulário 02: Exibe modificação somente após clicar no botão**

Estado Inicial

Vite + React + TS

localhost:5173/exemplo04

Trabalhando com HOOKS

- Home
- UseState
  - Exemplo01
  - Exemplo02
  - Exemplo03
  - Exemplo04

Exemplo 04 – Formulário 02: Exibe modificaçi

Nome: Mostrar Nome

Estado Final

Vite + React + TS

localhost:5173/exemplo04

Trabalhando com HOOKS

- Home
- UseState
  - Exemplo01
  - Exemplo02
  - Exemplo03
  - Exemplo04

Exemplo 04 – Formulário 02: Exibe modificação somen

Nome: Mostrar Nome

Nome digitado: Darth Vader

2.1

REACT – HOOKS – USESTATE – EXERCÍCIOS

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState – Exercícios**

1. **Alterar Texto ao Clicar no Botão**
   - ✓ Crie um componente Mensagem que tenha:
   - ✓ Um parágrafo exibindo um texto inicial ("Clique no botão para mudar a mensagem").
   - ✓ Um botão que, ao ser clicado, muda o texto para outra mensagem, como "Mensagem alterada!".
   - ✓ Se o botão for clicado novamente, o texto volta ao original (efeito "toggle").

   - ✓ **Objetivo**
     - ✓ Usar useState para controlar o estado de um texto.
     - ✓ Alternar valores dentro do estado com base em uma ação do usuário.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState – Exercícios**

2. Lista de Tarefas

- ✓ Crie um componente **ListaTarefas** que tenha:
- ✓ Um input para digitar o nome de uma tarefa.
- ✓ Um botão "Adicionar", que adiciona a tarefa à lista.
- ✓ Exibir as tarefas adicionadas na tela.
- ✓ Um botão "Remover" ao lado de cada tarefa, que a exclui da lista.

- ✓ **Objetivo**
  - ✓ Trabalhar com estados de array e atualização dinâmica de estado.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useState – Exercícios**

3. **Crie um componente CarrinhoDeCompras que tenha**:
   - ✓ Uma lista de produtos (nome e preço).
   - ✓ Um botão "Adicionar ao Carrinho" para cada produto.
   - ✓ Uma seção que mostra os produtos adicionados ao carrinho.
   - ✓ Um botão "Remover" ao lado de cada item do carrinho.
   - ✓ Um cálculo do total da compra baseado nos produtos do carrinho.

   - ✓ **Objetivo**
     - ✓ Gerenciar estados mais complexos com objetos e arrays no useState.

3

REACT – HOOKS – USEEFFECT

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

✓ HOOKS – useEffect

- ✓ O **useEffect** é um Hook do React que permite lidar com efeitos colaterais em componentes funcionais.
- ✓ Efeitos colaterais são operações que afetam algo fora do escopo da função, como modificar o DOM, buscar dados de uma API, configurar um timer ou adicionar event listeners.
- ✓ No React, usamos o **useEffect** para gerenciar esses efeitos de forma controlada.

```
import { useEffect } from "react";
    useEffect(() => {
        // Código a ser executado no efeito
    }
);
```

- ✓ Por padrão, o código dentro do useEffect **roda após a renderização** do componente.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useEffect**

  - **Quando utilizar?**
    - Você deve utilizar **useEffect** quando precisar realizar operações assíncronas, como chamadas de API, manipulação do DOM, gerenciamento de assinaturas de eventos ou qualquer operação que não seja parte do fluxo de renderização principal.
    - O **useEffect** é usado quando você precisa realizar efeitos colaterais, **como modificar o DOM**, fazer requisições HTTP, lidar com eventos globais (resize, keydown etc.) ou interagir com APIs externas. Ele roda **após a renderização** e pode ser controlado por dependências.
    - Chamadas de API → Buscar dados e atualizar o estado
    - Modificar o DOM fora do JSX → Exemplo: alterar document.title
    - Event listeners globais → resize, scroll, keydown, etc.
    - Timers e intervalos → setTimeout, setInterval
    - Subscrições → WebSockets, Firebase, etc.
    - Sincronizar estado com localStorage ou sessionStorage

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Quando NÃO utilizar?**
    - ✓ Para exibir ou atualizar o JSX com estados ou props (o React já faz isso automaticamente)
    - ✓ Para calcular valores derivados do estado (use useMemo ou diretamente no JSX)

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Exemplo 01 – Contador: Altera o título da página sempre que o estado count muda**

```
1.  import { useState, useEffect } from "react"; // Importa os Hooks useState e useEffect do React
2.  const UseEffect_Exemplo01 = () => {
3.   const [count, setCount] = useState(0);
4.   useEffect(() => {
5.    document.title = `Contagem: ${count}`; // Crase, significa que podemos inserir varáveis em uma string
6.   }, [count]); // Atualiza o título sempre que count mudar
7.   return (
8.    <>
9.     <h1>Exemplo 01 - Contador: Exibe modificação somente após clicar no botão</h1>
10.    <p>Você clicou {count} vezes</p>
11.     <button onClick={() => setCount(count + 1)}>Clique aqui</button>
12.    </>
13.   );
14. };
15. export default UseEffect_Exemplo01;
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Exemplo 01 – Contador: Altera o título da página sempre que o estado count muda**

Estado Inicial

Contagem: 0

localhost:5173/useEffect_exemplo01

Trabalhando com HOOKS

- Home
- useState
  - Exemplo01
  - Exemplo02
  - Exemplo03
  - Exemplo04
- useEffect
  - Exemplo01

Exemplo 01 - Contador: Exibe modifi

Você clicou 0 vezes

Clique aqui

Estado Final

Contagem: 4

localhost:5173/useEffect_exemplo01

Trabalhando com HOOKS

- Home
- useState
  - Exemplo01
  - Exemplo02
  - Exemplo03
  - Exemplo04
- useEffect
  - Exemplo01

Exemplo 01 - Contador: Exibe modificação somente após clicar no

Você clicou 4 vezes

Clique aqui

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Exemplo 02 – Mostra mensagens após um período de tempo**:

```
1.  import { useState, useEffect } from "react";
2.  const UseEffect_Exemplo01 = () => {
3.   import { useEffect, useState } from 'react';
4.   const UseEffect_Exemplo02 = () => {
5.   const frases = [
6.     "Frase 01: A persistência realiza o impossível.",
7.     "Frase 02: A prática leva à perfeição.",
8.     "Frase 03: Aprender é evoluir constantemente.",
9.     "Frase 04: Código limpo, mente limpa.",
10.  ];
11.  const [indice, setIndice] = useState(0);
```

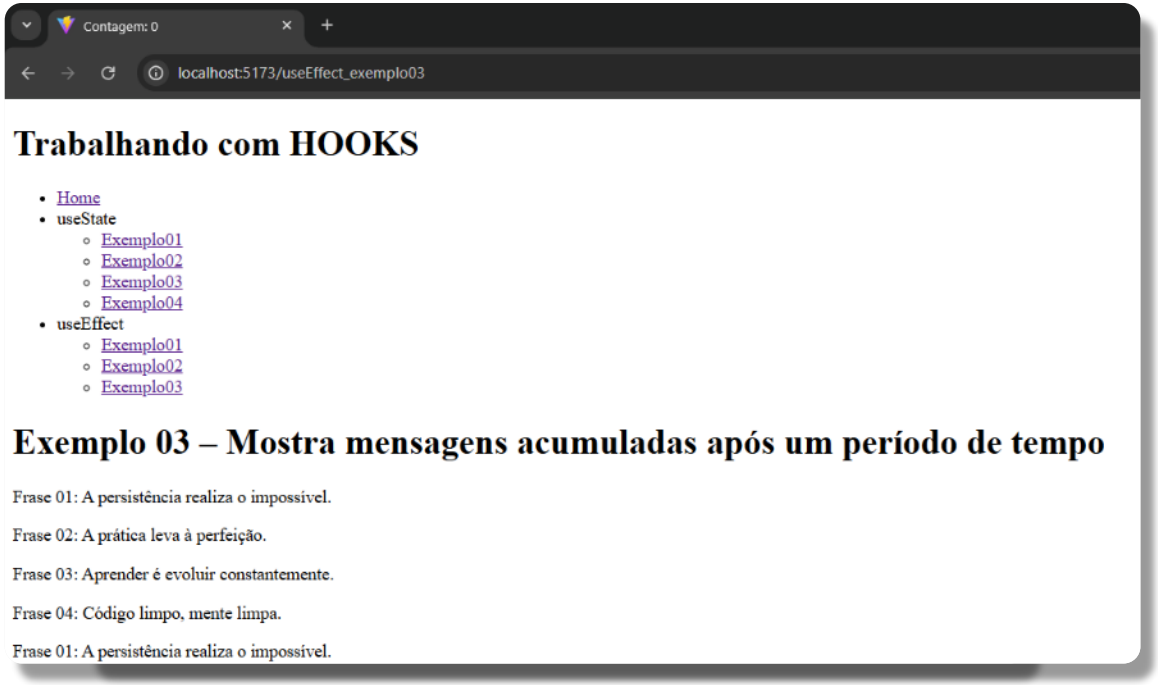
COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Exemplo 02 – Mostra mensagens após um período de tempo**:

```
12.   useEffect(() => {
13.     const intervalo = setInterval(() => {
14.       setIndice((indiceAtual) => (indiceAtual + 1) % frases.length);
15.     }, 3000); // troca a cada 3 segundos
16.       return () => clearInterval(intervalo); // limpeza ao desmontar
17.   }, [frases.length]);
18.   return (
19.   <>
20.     <h1>Exemplo 02 - Mostra mensagens após um período de tempo</h1>
21.     {frases[indice]}
22.   </>
23.   );
24.  };
     export default UseEffect_Exemplo02;
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Exemplo 02 – Mostra mensagens após um período de tempo**



COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Exemplo 03 – Mostra mensagens acumuladas após um período de tempo**:

```
1.  import { useState, useEffect } from "react";
2.  const UseEffect_Exemplo03 = () => {
3.    const frases = [
4.      "Frase 01: A persistência realiza o impossível.",
5.      "Frase 02: A prática leva à perfeição.",
6.      "Frase 03: Aprender é evoluir constantemente.",
7.      "Frase 04: Código limpo, mente limpa.",
8.    ];
9.    const [indice, setIndice] = useState(0);
10.   const [exibidas, setExibidas] = useState<string[]>([]);
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useEffect**
  - **Exemplo 03 – Mostra mensagens acumuladas após um período de tempo**:

```
11.  useEffect(() => {
12.    const intervalo = setInterval(() => {
13.      setExibidas((anteriores) => {
14.        const novaFrase = frases[indice];
15.        return [...anteriores, novaFrase];
16.      });
17.      setIndice((i) => (i + 1) % frases.length);
18.    }, 3000);
19.    return () => clearInterval(intervalo);
20.  }, [indice]);
```

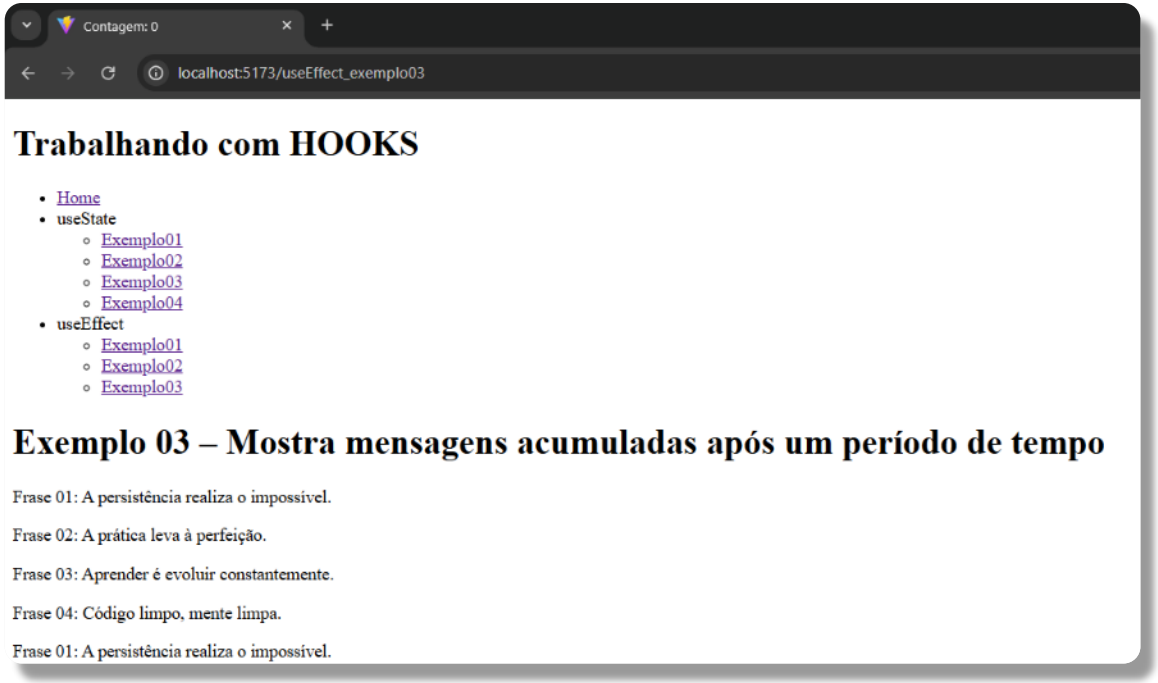
COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useEffect**
  - **Exemplo 03 – Mostra mensagens acumuladas após um período de tempo**:

```
21.   return (
22.     <>
23.       <h1>Exemplo 03 - Mostra mensagens acumuladas após um período de tempo</h1>
24.       {exibidas.map((frase, idx) => (
25.         <p key={idx}>{frase}</p>
26.       ))}
27.     </>
28.   );
29. };
30. export default UseEffect_Exemplo03;
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Exemplo 03 – Mostra mensagens acumuladas após um período de tempo**



COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Exemplo 04 – Detectar e exibir a largura da tela**:

```
1.  import { useState, useEffect } from "react";
2.  import { useState, useEffect } from 'react';
3.  const UseEffect_Exemplo04 = () => {
4.    const [largura, setLargura] = useState(window.innerWidth);
5.    useEffect(() => {
6.      const atualizarLargura = () => setLargura(window.innerWidth);
7.      window.addEventListener('resize', atualizarLargura);
8.      return () => {
9.        window.removeEventListener('resize', atualizarLargura);
10.     };
11.   }, []);
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

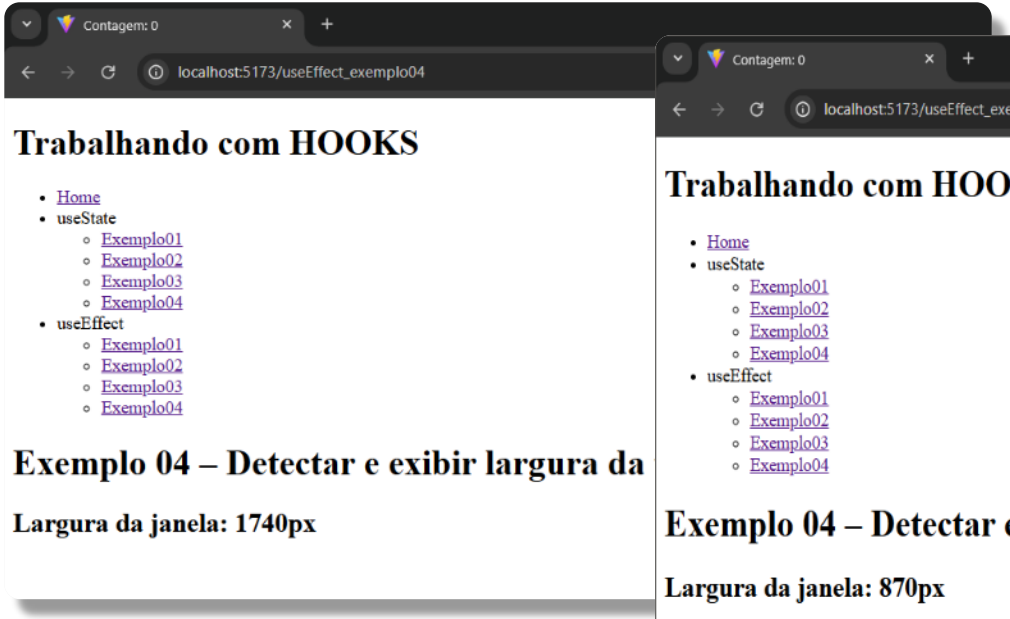
- **HOOKS – useEffect**
  - **Exemplo 04 – Detectar e exibir a largura da tela**:

```
12.    return (
13.      <>
14.        <h1>Exemplo 03 - Detectar e exibir largura da tela</h1>
15.        <h2>Largura da janela: <span>{largura}px</span></h2>
16.      </>
17.    );
18.  };
19.  export default UseEffect_Exemplo04;
```

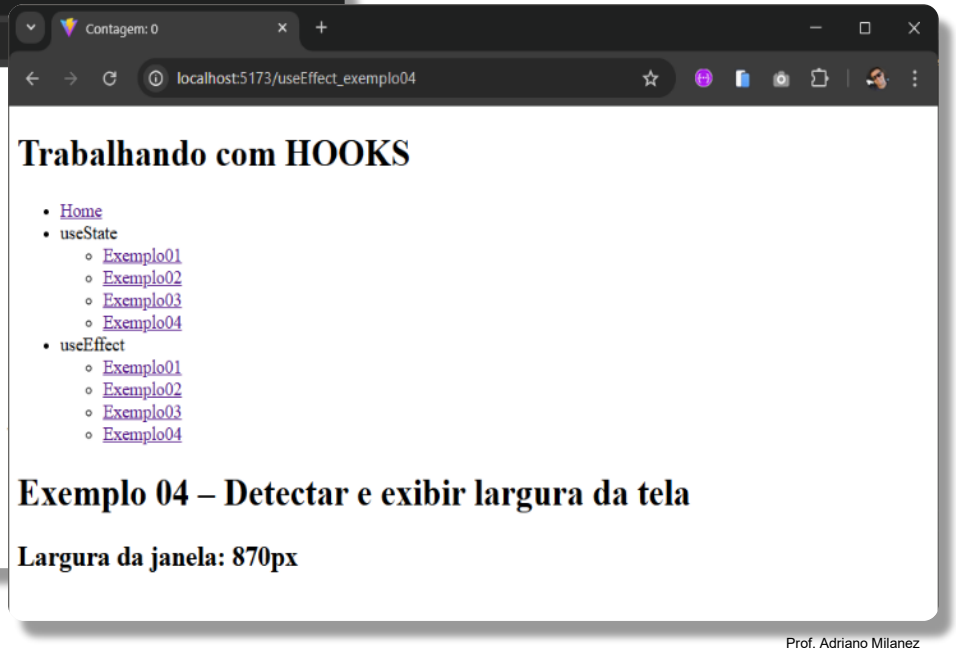
COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect**
  - ✓ **Exemplo 04 – Detectar e exibir a largura da tela**

Estado Inicial



Estado Final



3.1

REACT – HOOKS – USEEFFECT – EXERCÍCIOS

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect – Exercícios**

1. **Mudar a cor do fundo automaticamente**
   - ✓ Crie um componente que altere a cor de fundo da página a cada 3 segundos.
   - ✓ **Requisitos**:
     - ✓ O componente deve alterar a cor de fundo do body a cada 3 segundos.
     - ✓ As cores devem ser escolhidas aleatoriamente
     - ✓ Use useEffect para iniciar o efeito quando o componente for montado
     - ✓ Dica: Você pode usar setInterval e Math.random() para gerar cores

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect – Exercícios**

2. Detectar teclas pressionadas

- ✓ Crie um componente que detecte quando o usuário pressiona a tecla "Enter" e exiba uma mensagem na tela.
- ✓ **Requisitos**:
  - ✓ Use useEffect para adicionar um event listener para keydown
  - ✓ Se o usuário pressionar "Enter", exiba "Você pressionou Enter!“
  - ✓ Remova o event listener quando o componente for desmontado

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useEffect – Exercícios**

3. **Buscar dados e atualizar automaticamente**:
   - ✓ Crie um componente que exiba uma lista de posts de uma API e atualize os dados a cada 10 segundos.
   - ✓ **Requisitos**
     - ✓ Faça uma requisição para a API https://jsonplaceholder.typicode.com/posts
     - ✓ Exiba os títulos dos primeiros 5 posts
     - ✓ Atualize a lista automaticamente a cada 10 segundos usando setInterval
     - ✓ Remova o intervalo ao desmontar o componente

REACT – HOOKS – USECONTEXT

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useContext**

  - O **useContext** é um hook do React que permite compartilhar dados entre componentes sem precisar passar props manualmente em todas as camadas. É ideal para estados globais ou compartilhados — como temas, autenticação, idioma, etc.

  - **Como funciona?**
    - Cria-se um contexto com createContext
    - Um provedor (Provider) fornece o valor do contexto
    - Os componentes filhos acessam o contexto usando useContext

  - **Quando usar useContext?**
    - Quando vários componentes precisam acessar o mesmo dado.
    - Quando você quer evitar "prop drilling" (passar props de pai para filho até chegar no componente final).
    - Para gerenciar temas (light/dark), autenticação (usuário logado), carrinho de compras, etc.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useContext**

  - O **useContext** é um hook do React que permite compartilhar dados entre componentes sem precisar passar props manualmente em todas as camadas. É ideal para estados globais ou compartilhados — como temas, autenticação, idioma, etc.

  - **Como funciona?**
    - Cria-se um contexto com createContext
    - Um provedor (Provider) fornece o valor do contexto
    - Os componentes filhos acessam o contexto usando useContext

  - **Quando usar useContext?**
    - Quando vários componentes precisam acessar o mesmo dado.
    - Quando você quer evitar "prop drilling" (passar props de pai para filho até chegar no componente final).
    - Para gerenciar temas (light/dark), autenticação (usuário logado), carrinho de compras, etc.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useContext**

  - ✓ Caso você tenha criado o projeto e não instalado o **TAILWINDCSS**, retorne a **aula sobre estilização** e no **Item 3 TAILWINDCSS**, e faça os **passos 8 a 10**, sendo que no **passo 10**, vamos editar o arquivo **index.css** com o código abaixo.
  - ✓ Caso já tenha criado o projeto utilizando **TAILWINDCSS**, apenas modifique o arquivo **index.css** para o código abaixo

```
1.  body {
2.    @apply bg-white text-black;
3.  }
4.  .dark body {
5.    @apply bg-gray-800 text-white transition-colors duration-300;
6.  }
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useContext**
  - ✓ **Exemplo 01 – Alterar Tema para DARK | LIGHT**
    - ✓ Criando o **src/context/ThemeProvider.tsx**

```
1.  // src/context/ThemeProvider.tsx
    import { useState, useEffect, type ReactNode } from "react";
2.  import { ThemeContext, type Theme } from "./ThemeContext";
3.  export const ThemeProvider = ({ children }: { children: ReactNode }) => {
4.    const [theme, setTheme] = useState<Theme>("light");
5.    useEffect(() => {
6.      const stored = localStorage.getItem("theme") as Theme;
7.      if (stored) setTheme(stored);
8.    }, []);
9.    useEffect(() => {
10.     document.documentElement.classList.toggle("dark", theme === "dark");
11.     localStorage.setItem("theme", theme);
12.   }, [theme]);
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useContext**
  - ✓ **Exemplo 01 – Alterar Tema para DARK | LIGHT**
    - ✓ Criando o **src/context/ThemeProvider.tsx**

```
13.   const toggleTheme = () => {
14.     setTheme((prev) => (prev === "light" ? "dark" : "light"));
15.   };
16.   return (
17.     <ThemeContext.Provider value={{ theme, toggleTheme }}>
18.       {children}
19.     </ThemeContext.Provider>
20.   );
21. };
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useContext**
  - ✓ **Exemplo 01 – Alterar Tema para DARK | LIGHT**
    - ✓ Criando o **src/context/ThemeContext.tsx**

```
1.  // src/context/ThemeContext.tsx
    import { createContext } from "react";
2.  export type Theme = "light" | "dark";
3.  interface ThemeContextType {
4.    theme: Theme;
5.    toggleTheme: () => void;
6.  }
7.  export const ThemeContext = createContext<ThemeContextType>({
8.    theme: "light",
9.    toggleTheme: () => {},
10. });
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useContext**
  - **Exemplo 01 – Alterar Tema para DARK | LIGHT**
    - Modificando o arquivo **src/App.tsx**

```
1.  import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
2.  import UseState_Exemplo01 from "./pages/UseState/UseState_Exemplo01";
…   …
7.  import UseEffect_Exemplo01 from "./pages/UseEffect/UseEffect_Exemplo01";
…   …
12. import { useContext } from 'react';
13. import { ThemeContext } from "./context/ThemeContext";
14. const App = () => {
15.   const { theme, toggleTheme } = useContext(ThemeContext);
16.   return (
17.     <BrowserRouter>
18.       <div>
19.         <button onClick={toggleTheme}>Alternar Tema</button>
20.       </div>
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

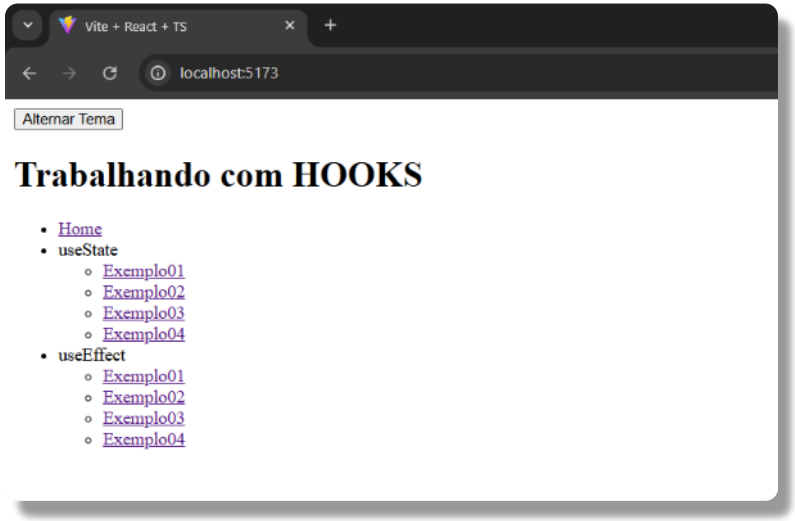
- **HOOKS – useContext**
  - **Exemplo 01 – Alterar Tema para DARK | LIGHT**
    - Modificando o arquivo **src/main.tsx**

```
1.  import React from 'react';
2.  import ReactDOM from 'react-dom/client';
3.  import App from './App';
4.  import './index.css';
5.  import { ThemeProvider } from './context/ThemeProvider';
6.  ReactDOM.createRoot(document.getElementById('root')!).render(
7.    <React.StrictMode>
8.      <ThemeProvider>
9.        <App />
10.     </ThemeProvider>
11.   </React.StrictMode>
12. );
```

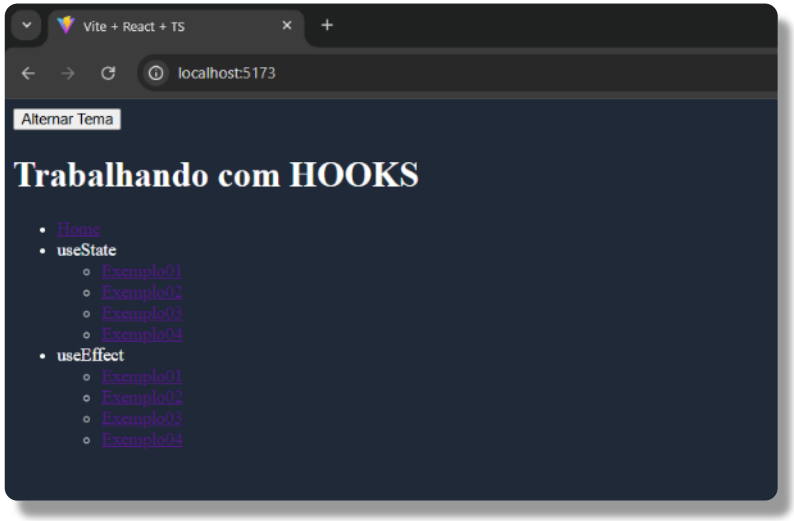
COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useContext**
  - ✓ **Exemplo 01 – Alterar Tema**

Estado Inicial



Estado Final



COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useContext**
  - **Exemplo 02 –** Criar o componente para compartilhamento do **nome de usuário**:
    - Vamos o componente **src/context/UserContext.tsx**

```
1.  import { createContext } from "react";
2.  export interface UserContextType {
3.   username: string;
4.   setUsername: (name: string) => void;
5.  }
6.  export const UserContext = createContext<UserContextType>({
7.   username: "Visitante",
8.   setUsername: () => {},
9.  });
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useContext**
  - **Exemplo 02 –** Criar o componente para compartilhamento do **nome de usuário**:
    - Vamos o componente **src/context/UserProvider.tsx**

```
1.  import { useState, type ReactNode } from "react";
2.  import { UserContext } from "./UserContext";
3.  export const UserProvider = ({ children }: { children: ReactNode }) => {
4.    const [username, setUsername] = useState("Visitante");
5.    return (
6.      <UserContext.Provider value={{ username, setUsername }}>
7.        {children}
8.      </UserContext.Provider>
9.    );
10. };
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useContext**
  - **Exemplo 02 –** Criar o componente para compartilhamento do **nome de usuário**:
    - Vamos modificar o componente **App.tsx**

```
1.   import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
2.   import UseState_Exemplo01 from "./pages/UseState/UseState_Exemplo01";
…    …
7.   import UseEffect_Exemplo01 from "./pages/UseEffect/UseEffect_Exemplo01";
…    …
12.  import Profile from "./pages/Profile/Profile";
…    …
16.  import { UserContext } from "./context/UserContext";
…    …
21.      const { username } = useContext(UserContext);
…        …
31.      <p>Usuário atual: { username }</p>
…     …
88.      <Route path="/profile" element={ <Profile /> } />
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useContext**
  - **Exemplo 02 –** Criar o componente para compartilhamento do **nome de usuário**:
    - Vamos modificar o componente **main.tsx**

```
1.  import React from 'react';
2.  import ReactDOM from 'react-dom/client';
3.  import App from './App';
4.  import './index.css';
5.  import { ThemeProvider } from './context/ThemeProvider';
6.  import { UserProvider } from "./context/UserProvider";
7.  ReactDOM.createRoot(document.getElementById('root')!).render(
8.    <React.StrictMode>
9.      <ThemeProvider>
10.       <UserProvider>
11.         <App />
12.       </UserProvider>
13.     </ThemeProvider>
14.   </React.StrictMode>
15. );
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- **HOOKS – useContext**
  - **Exemplo 02 –** Criar o componente para compartilhamento do **nome de usuário**:
    - Vamos criar o componente **pages/profile.tsx**

```
1.  import { useContext, useState } from "react";
2.  import { UserContext } from "../context/UserContext";
3.  const Profile = () => {
4.    const { username, setUsername } = useContext(UserContext);
5.    const [input, setInput] = useState(username);
6.    const handleSubmit = (e: React.FormEvent) => {
7.      e.preventDefault();
8.      setUsername(input);
9.    };
```

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

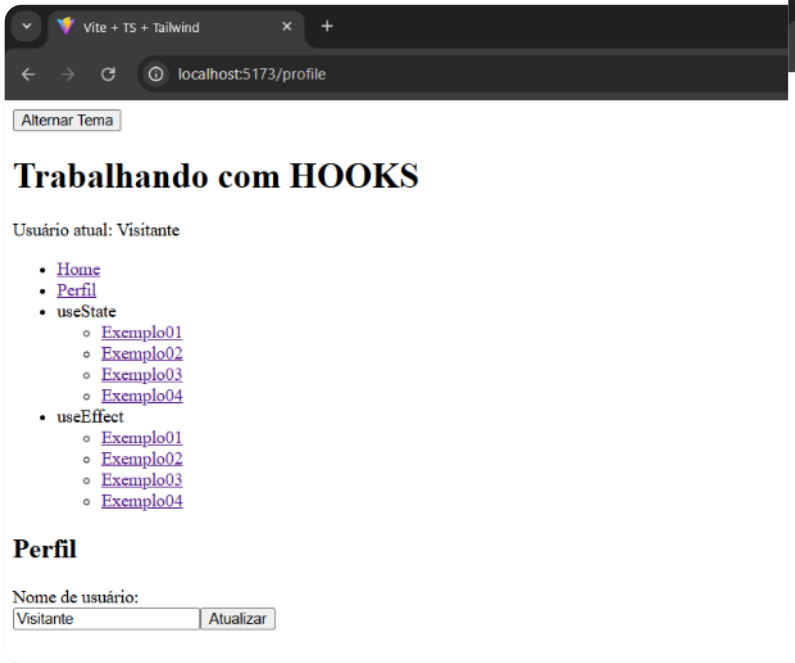
- **HOOKS – useContext**
  - **Exemplo 02 –** Criar o componente para compartilhamento do **nome de usuário**:
    - Vamos criar o componente **pages/profile.tsx**

```
11.     <div>
12.       <h2>Perfil</h2>
13.       <form onSubmit={handleSubmit}>
14.         <label htmlFor="username">Nome de usuário:</label><br />
15.         <input id="username" type="text" value={input} onChange={(e) => setInput(e.target.value)} />
16.         <button type="submit">Atualizar</button>
17.       </form>
18.       <p>Usuário atual: {username}</p>
19.     </div>
20.   );
21. };
    export default Profile;
```

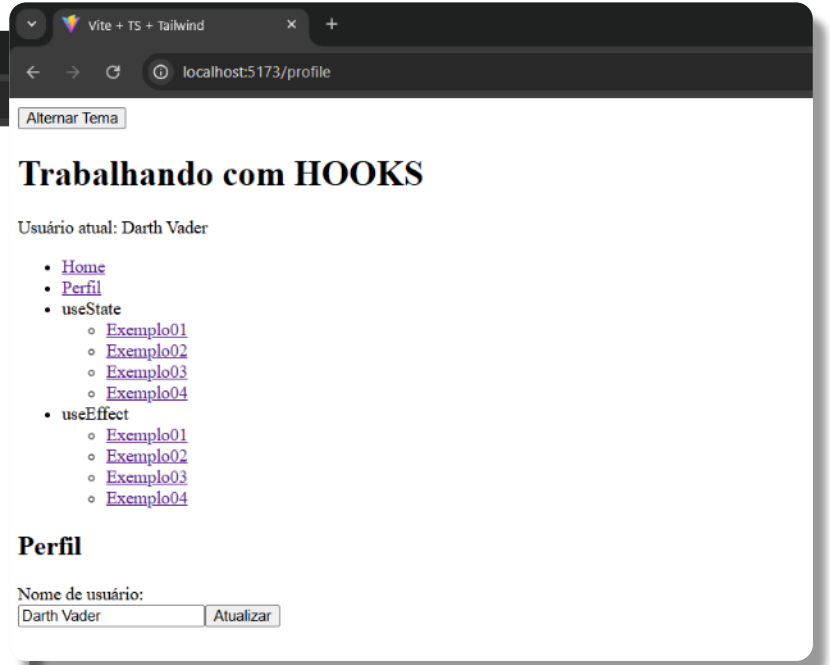
COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useContext**
  - ✓ **Exemplo 02 –** Criar um contexto para armazenar e exibir o nome de um usuário sem precisar passar **props** manualmente:

Estado Inicial



Estado Final



4.1

REACT – HOOKS – USECONTEXT – EXERCÍCIOS

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useContext – Exercícios**

1. **Controle de Autenticação do Usuário**
   - ✓ Criar um contexto para gerenciar o estado de autenticação do usuário (logado ou não).
   - ✓ **Passos:**
     - ✓ Crie um AuthContext que armazena o estado de autenticação (isAuthenticated como true ou false).
     - ✓ Crie um componente LoginButton.tsx que altera o estado de autenticação (logar/deslogar).
     - ✓ Exiba uma mensagem no componente UserStatus.tsx que informa se o usuário está logado ou não.
     - ✓ O botão de login deve alternar entre "Login" e "Logout", dependendo do estado de autenticação.
   - ✓ **Arquivos:**
     - ✓ AuthContext.tsx (fornece o estado de autenticação)
     - ✓ LoginButton.tsx (botão para login/logout)
     - ✓ UserStatus.tsx (exibe o status do usuário)
     - ✓ Home.tsx (exibe os componentes)
   - ✓ **Saída esperada:**
     - ✓ O usuário vê "Você está logado" ou "Você não está logado".
     - ✓ O botão alterna entre "Login" e "Logout" de acordo com o estado de autenticação.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useContext – Exercícios**

  2. **Gerenciamento de Idioma**

     - ✓ Criar um contexto para mudar o idioma da aplicação (Português/Inglês).
     - ✓ **Passos:**
       - ✓ Crie um LanguageContext que armazena o idioma ("pt-br" ou "en").
       - ✓ Crie um componente LanguageSelector.tsx que exibe um botão para trocar o idioma.
       - ✓ Crie um WelcomeMessage.tsx que exibe "Bem-vindo!" em português e "Welcome!" em inglês.
     - ✓ **Arquivos:**
       - ✓ LanguageContext.tsx (fornece o idioma)
       - ✓ LanguageSelector.tsx (muda o idioma)
       - ✓ WelcomeMessage.tsx (exibe mensagem no idioma correto)
       - ✓ Home.tsx (exibe os componentes)
     - ✓ **Saída esperada:**
       - ✓ O usuário pode alternar entre "Bem-vindo!" e "Welcome!" clicando no botão.

COMO OS HOOKS FACILITAM A VIDA DO DESENVOLVEDOR

- ✓ **HOOKS – useContext – Exercícios**

  3. **Carrinho de Compras**:
     - ✓ Criar um contexto para gerenciar um carrinho de compras.
     - ✓ **Passos:**
       - ✓ Crie um CartContext que armazena os produtos do carrinho.
       - ✓ Crie um ProductList.tsx que exibe produtos e tem um botão "Adicionar ao Carrinho".
       - ✓ Crie um Cart.tsx que exibe os produtos adicionados e um botão "Remover".
       - ✓ Atualize a contagem de itens ao adicionar ou remover produtos.
     - ✓ **Arquivos:**
       - ✓ CartContext.tsx (fornece o carrinho)
       - ✓ ProductList.tsx (lista de produtos)
       - ✓ Cart.tsx (exibe os produtos adicionados)
       - ✓ Home.tsx (exibe os componentes)
     - ✓ **Saída esperada:**
       - ✓ O usuário pode adicionar e remover produtos do carrinho, e a quantidade é atualizada dinamicamente.

Referências Bibliográficas

ATEL, Vishal Layka. Pro TypeScript: Application-Scale JavaScript Development. 1. ed. Berkeley: Apress, 2014.

BANKS, Adam. Learning React: Functional Web Development with React and Redux. 2. ed. Sebastopol: O'Reilly Media, 2020.

CEDDIA, Dave. Pure React: A step-by-step guide to mastering React. 1. ed. [S.l.]: Self-published, 2018.

CSS-TRICKS. Using the State Hook in React. Disponível em: https://css-tricks.com/using-the-state-hook-in-react/. Acesso em: 19 de julho de 2025.

DEV.TO. What are React Hooks?. Disponível em: https://dev.to (buscar por "react hooks"). Acesso em: 19 de julho de 2025.

DIGITALOCEAN. Understanding React Hooks. Disponível em: https://www.digitalocean.com/community/tutorials/understanding-react-hooks. Acesso em: 19 de julho de 2025.

FERNANDES, Flavio. Desenvolvimento Front-end com React e TypeScript. 1. ed. São Paulo: Novatec Editora, 2021.

FREECODECAMP. React Hooks: A Complete Introduction. Disponível em: https://www.freecodecamp.org/news/react-hooks-introduction/. Acesso em: 19 de julho de 2025.

FREEMAN, Eric; ROBSON, Elisabeth. Head First JavaScript Programming. 1. ed. Sebastopol: O'Reilly Media, 2014.

GOLDBERG, Josh. Aprendendo TypeScript: Melhore Suas Habilidades de Desenvolvimento web Usando JavaScript Type-Safe. São Paulo: Novatec. 2022

HOLOWAYCHUK, TJ. Modern JavaScript. 1. ed. [S.l.]: Independently published, 2022.

LOGROCKET. A Guide to useEffect Hook in React. Disponível em: https://blog.logrocket.com/a-guide-to-useeffect-hook-in-react/. Acesso em: 19 de julho de 2025.

MAAT, Cassio de Sousa. React.js Essencial: Um guia para o desenvolvedor JavaScript. 1. ed. São Paulo: Casa do Código, 2020.

MARDER, Carl. React and React Native. 3. ed. Birmingham: Packt Publishing, 2020.

MDN WEB DOCS. CSS3 - Desenvolva aplicações web profissionais com o uso dos poderosos recursos de Hooks das CSS. São Paulo: Novatec Editora, 2010.

_____. Guia JavaScript. Disponível em <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide>. Acessado em 19 de julho de 2025.

META. Hooks – React. Disponível em: https://react.dev/learn/hooks. Acesso em: 19 de julho de 2025.

NELSON, J. Learn React's Fundamentals Without the Buzzwords? 2018. Disponível em: <jamesknelson.com/learn-react-fundamentals-sans-buzzwords>. Acesso em: 19 de julho de 2025.

Referências Bibliográficas

RAUSCHMAYER, A. Speaking JavaScript: An In-Depth Guide for Programmers. Estados Unidos: O'Reilly Media, 2014.

REACTIVA. O arquivo package-lock.json. Disponível em: <https://nodejs.reativa.dev/0020-package-lock-json/index>. Acessado em 19 de julho de 2025.

REACTJS.ORG. React Context API. Disponível em: https://reactjs.org/docs/context.html. Acesso em: 19 de julho de 2025.

RICOY, L. Desmitificando React: Uma Reflexão para Iniciantes. 2018. Disponível em: <medium.com/trainingcenter/desmitificando-react-uma-reflex%C3%A3o-para-iniciantes-a57af90b6114>. Acesso em: 19 de julho de 2025.

RIEDEL, Alex. TypeScript Quickly. 1. ed. Shelter Island: Manning Publications, 2020.P

STACK OVERFLOW. React Hooks Examples. Disponível em: https://stackoverflow.com/questions/tagged/react-hooks. Acesso em: 19 de julho de 2025.

SWC. Rust-based platform for the Web. 2024 Disponível em <https://swc.rs/>. Acessado em 19 de julho de 2025.

TUTORIALSPOINT. ReactJS - useContext Hook. Disponível em: https://www.tutorialspoint.com/reactjs/reactjs_usecontext_hook.htm. Acesso em: 19 de julho de 2025.

VITE. The React Framework for the Web. Disponível em: https://VITE.org/. Acesso em: 19 de julho de 2025.

W3SCHOOLS. React Context API. Disponível em: https://www.w3schools.com/react/react_context.asp. Acesso em: 19 de julho de 2025.

_____. React Hooks. Disponível em: https://www.w3schools.com/react/react_hooks.asp. Acesso em: 19 de julho de 2025.

WILSON, Michael. Modern Full-Stack Development: Using TypeScript, React, Node.js, Webpack, and Docker. 1. ed. Sebastopol: O'Reilly Media, 2020.

Dúvidas?

Críticas?

Sugestões?

Ameaças?